MagTapp Founding Engineer Technical Assignment

MagTapp Architecture

Backend structure

Modular monolith at this scale — one deployable service split into clean domains (auth, users, content, subscriptions, ai, notifications), each owning its own tables. Fast iteration, no distributed-systems tax, and a natural seam to peel off ai later once its GPU/latency profile diverges. Built in Go (preferred, for concurrency/throughput on the browsing+AI-proxy layer) — TypeScript as a fallback if the team is stronger there.

Why not Python??
Python (FastAPI/Django) is fine for prototyping but weaker here for a browser-proxy/AI-gateway workload - the GIL limits true concurrency for I/O-heavy fan-out (many simultaneous AI/content requests).
Go's goroutines or Node's event loop handle that pattern more efficiently and cheaply at scale. Python also typically needs more infra (Gunicorn workers, Celery for async) to match what Go/Node give you natively, meaning more moving parts for a small founding team to run.

Auth & Limits

Firebase Auth for sign-in (Google/Apple/email-OTP) — issues JWTs, verified at the API edge via Firebase Admin SDK middleware. Redis holds refresh-token/session state, revocation lists, and per-user rate limits. Casbin (RBAC/ABAC) enforces authorization rules (free vs Pro, admin vs user) at the service layer using the verified JWT claims. Postgres users table keyed by UUID, mapped to Firebase uid, with anonymous device-based users merged into real accounts on signup.

API structure

Versioned endpoints (/v1, /v2), with old versions kept live and fully supported until analytics confirm 100% client migration to the latest — never break a shipped mobile client since app store rollout is slow. Sunset old versions only after a deprecation window + forced-upgrade banner in-app.

Scalability & reliability

Blue-green deployments for zero-downtime releases and instant rollback if the new version misbehaves. For cost predictability, consider fixed-price VPS (e.g., Hetzner/DigitalOcean) for steady-state baseline load instead of pure AWS pay-as-you-go, since VPS gives constant pricing rather than usage-based bills that spike with traffic — reserving AWS (or burstable instances) only for elastic/peak overflow.

One honest flag: the assignment explicitly asks "which AWS services would you consider and why," so in your actual submission I'd keep AWS as the primary answer for that section. But you can add a short note like "for cost control, a hybrid approach like fixed-price VPS for baseline compute, AWS for elastic/managed services (RDS, S3, Cognito) which can reduce PAYG variance", that shows cost-awareness without ignoring what they asked for.

Deployment & Cost

Need	AWS Service	Self-hosted/OSS Alternative	Why
Compute	ECS Fargate	Docker + Docker Compose / Nomad, orchestrated behind a reverse proxy	Full control, no per-vCPU billing; Nomad if you outgrow Compose but don't want K8s complexity
DB	RDS Postgres (Multi-AZ)	Self-managed PostgreSQL with Patroni + repmgr for HA, or plain streaming replication	Same relational guarantees; Patroni gives automated failover across VPS nodes
Cache	ElastiCache Redis	Redis (or KeyDB for better multi-threaded perf) self-hosted, with Sentinel for failover	Identical feature set, no managed markup
Object storage	S3	MinIO (S3-compatible API) or Backblaze	Drop-in replacement — same SDK calls, self-hosted, erasure-coded for durability
CDN	CloudFront	Cloudflare (free/pro tier) or BunnyCDN in front of your VPS origin	Cheap/free edge caching + DDoS protection without owning edge infra yourself
Auth	Cognito	Casbin or Keycloak or Ory (Kratos/Hydra)	Open-source identity providers, OIDC/JWT-compatible, self-hosted
API edge	API Gateway/ ALB	NGINX / Traefik / Kong (OSS edition)	Routing, TLS termination, rate limiting, auth middleware
Async/ events	SQS + EventBridge	RabbitMQ or Redis Streams / NATS	Message queuing and event-driven decoupling, self-

			hosted
Observability	CloudWatch + X-Ray	Prometheus + Grafana + Loki, and Jaeger/Tempo for tracing	Full metrics/logs/traces stack, OSS, no per-GB ingestion cost
Secrets	Secrets Manager	HashiCorp Vault (OSS) or SOPS + age for encrypted config files	Centralized secret storage with access policies
IaC	Terraform/ CDK	Terraform (still works — has providers for bare-metal/ Proxmox/Hetzner) + Ansible for config management	Terraform for provisioning, Ansible for server config — both OSS

2. Launching MagTapp Pro

Overview: the complete journey

User taps Subscribe Platform Billing (StoreKit / Play Billing), payment handled by Apple/Google App receives receipt/purchase token App Backend: POST /v1/subscriptions/verify (with Idempotency-Key header) Backend verifies token server-to-server with Apple/Google Postgres (source of truth) updated + Redis entitlement cache refreshed Signed entitlement claim returned App unlocks Pro Apple/Google webhooks (renew/fail/expire/refund) Backend keeps entitlement in sync

iOS & Android

Must use native billing (StoreKit 2 / Google Play Billing via Flutter's in_app_purchase), mobile subscriptions cannot route through a custom payment gateway; Apple/Google reject apps that try.

Backend verification

Client-reported status is never trusted. Backend independently verifies the receipt/token against Apple's App Store Server API / Google's Play Developer API before granting entitlement. *(RevenueCat alternative: offloads this verification step - their SDK/webhook tells your backend the result instead of you calling Apple/Google directly.)*

Idempotency & failure safety

- Every verify and webhook call carries an **idempotency key** (Apple/Google's transaction_id / notification_id), backend stores processed keys and short-circuits duplicate/retried requests without double-writing entitlement or double-firing side effects (emails, analytics).
- **Webhook signature verification** (Apple JWS / Google Pub/Sub auth) before trusting any payload, prevents forged "subscription active" calls.
- **Nightly reconciliation job**: re-checks subscriptions near expiry directly against Apple/Google, as a safety net for missed/delayed webhooks.
- **At-least-once webhook handling**: writes are structured as upserts keyed on original_transaction_id, so a retried webhook is safe to replay.

Where status lives

- **Postgres** = source of truth (subscriptions table: user_id, platform, status, current_period_end, original_transaction_id).
- **Redis** = fast-read entitlement cache, invalidated on every webhook/verify update.
- App never self-reports entitlement - always reflects backend state, cached locally only briefly for offline UX.

Payment failure

Apple/Google auto-retry billing (grace period, ~up to 16 days). Backend marks grace_period on failure webhook; on final failure, expired → cache invalidated → app downgraded. Soft in-app prompt ("update payment method") rather than an immediate hard lock.

Renewal & expiry

Driven by platform webhooks: DID_RENEW, EXPIRED, GRACE_PERIOD, REFUND each update Postgres + invalidate Redis. Refund revokes entitlement immediately regardless of period end.

Restore on another device

Entitlement is tied to the **MagTapp account**, not the device. User signs in → app calls platform restore → sends token to /verify → backend matches original_transaction_id to the existing subscription and re-links it to the current user_id. No double charge, since it's the same underlying platform subscription.

Preventing client manipulation

1. Server-side entitlement check on every Pro-gated endpoint - never a client-sent flag.
2. Entitlement delivered as a short-lived **signed JWT claim**, not a locally-stored boolean.
3. Server-to-server receipt verification only.
4. API-level gating, not just UI - a patched client still gets a 403.
5. Root/jailbreak detection as a soft signal; anomaly detection for one token reused across many accounts.
6. Signed/verified webhooks only.

Note on pricing

Treated ₹99 / ₹299 as two price points for one Pro tier (e.g. regional/promo vs standard), not two feature tiers - if two real tiers are wanted, same design extends with separate product IDs.

3. AWS Cost & Production Thinking

Week 1: Inspect first

- **Cost Explorer + Billing dashboard**: break down spend by service, tag, and day/week trend, find the top 3-5 cost drivers immediately.
- **AWS Trusted Advisor / Compute Optimizer**: flags idle/underutilized resources out of the box.
- **Architecture review**: list every running service (EC2/ECS, RDS, S3, CloudFront, NAT Gateway, Lambda) and map which app component owns it, orphaned resources from old experiments are common.

Services/cost categories to investigate

- **Compute**: over-provisioned EC2/Fargate (low CPU/memory utilization over 2+ weeks), dev/staging instances left running 24/7.
- **Data transfer / NAT Gateway**: often a silent, disproportionate cost: cross-AZ traffic, NAT egress for AI API calls.
- **RDS**: idle read replicas, oversized instance classes, un-reclaimed storage.
- **S3**: missing lifecycle policies (old logs/exports never moved to Glacier or deleted).
- **CloudWatch logs**: unbounded retention on verbose logs.

Metrics/logs to check

CPU/memory utilization, request latency & error rate (p50/p95/p99), RDS connection count, Redis eviction rate, S3 storage growth, NAT/data-transfer bytes, and CloudWatch cost-anomaly alerts.

Identifying unnecessary usage

Correlate low utilization + steady billing over time → candidates for downsizing or termination. Tag resources by owner/environment to spot forgotten dev/test infra. Compute Optimizer's recommendations as a cross-check.

Making changes safely

- Change one thing at a time, in staging first, with rollback ready.
- Right-size non-critical (dev/staging) resources first, lowest blast radius.
- Use blue-green or canary rollout for any production compute change.
- Never touch DB instance class/storage without a tested backup + maintenance window.

Monitoring/alerts to introduce

- CloudWatch billing alarms (daily spend threshold).
- Utilization-based alerts (sustained low CPU → downsizing candidate).
- Error rate / latency alerts on the API layer.
- Cost anomaly detection (AWS Cost Anomaly Detection) for sudden spend spikes.

Longer-term recommendation: shift to VPS

AWS PAYG pricing is efficient for spiky/uncertain load, but MagTapp already has steady, predictable traffic (6M+ downloads, established MAU), at that point, PAYG compute/data-transfer costs compound fast, and much of that spend is now paying for AWS's flexibility rather than for capacity actually needed.

Moving baseline compute + DB + cache to **fixed-price VPS/bare-metal** (Hetzner, OVH, or similar) trades a harder initial setup (self-managed HA, patching, no managed-service convenience) for **flat, predictable pricing**, a VPS's monthly cost can often cover what AWS bills in a single week at

similar load, meaning the yearly VPS bill can undercut a *month* of AWS PAYG at scale.

Practical path: keep AWS (or Cloudflare) for CDN/edge and burst overflow only, migrate steady-state compute/DB/cache to VPS first (lowest risk, biggest savings), and treat the harder setup cost as a one-time investment that pays back within the first couple of months at MagTapp's traffic level.